

```

1 import rebound
2 import numpy as np
3 import matplotlib.pyplot as plt
4 from matplotlib.ticker import AutoMinorLocator
5 ✓ [1] 650毫秒

1 sim = rebound.Simulation()
2 sim.units = ('yr', 'AU', 'Msun')
3 sigma = 5.67037442e-08 * (365.25*86400) * (1.49597871e+11)**2 # J/(yr*AU**2*K**4)
4 m_A = 1.0479
5 R_A = 1.1618*695700000/1.49597871e+11
6 m_B = 1.0208
7 R_B = 1.0927*695700000/1.49597871e+11
8 m_P = 0.220*1.8981246e+27/1.98840987e+30
9 R_P = 0.764*71492000/1.49597871e+11
10 a_P = 1.0896
11 e_P = 0.182
12 i_P = 90.355
13
14 M = m_A + m_B
15 a_sep = 0.22882
16 a_A = (m_B / M) * a_sep
17 a_B = (m_A / M) * a_sep
18 omega = np.sqrt(sim.G*M/a_sep**3)
19
20 v_A = omega*a_A
21 v_B = omega*a_B
22 sim.add(m=m_A, x=-a_A, y=0, z=0, vx=0, vy=-v_A, vz=0)
23 sim.add(m=m_B, x=a_B, y=0, z=0, vx=0, vy=v_B, vz=0)
24
25 v_P = np.sqrt(sim.G*(M+m_P)/(a_P*(1-e_P**2)))*(1+e_P)
26 sim.add(m=m_P, x=a_P*(1-e_P), y=0, z=0, vx=0, vy=v_P, vz=0)
27
28 op = rebound.OrbitPlot(sim, figsize=(6,6), unitlabel="[AU]", color=True, periastron=True, xlim=[-1.5,1.0], ylim=[-1.25,1.25])
29 op.fig.savefig("orbit.png", format='png', dpi=300, bbox_inches='tight', pad_inches=0.2)

```

```

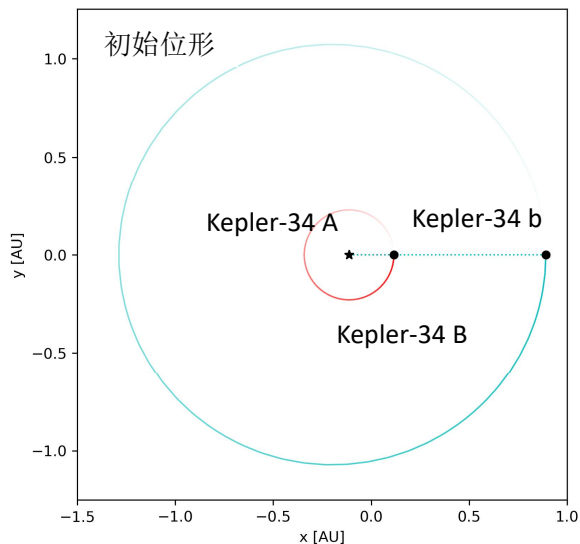
1 sim.integrator = "ias15"
2 dt = 0.00001
3 t_end = 10.0
4 n_steps = int(t_end/dt)
5
6 radiation_intensity = np.zeros(n_steps)
7 equilibrium_temperature = np.zeros(n_steps)
8
9 for i in range(n_steps):
10     sim.integrate(sim.t + dt)
11
12     r_A = np.array(sim.particles[0].xyz)
13     r_B = np.array(sim.particles[1].xyz)
14     r_P = np.array(sim.particles[2].xyz)
15
16     r_AB = np.linalg.norm(r_B - r_A)
17     r_AP = np.linalg.norm(r_P - r_A)
18     r_BP = np.linalg.norm(r_P - r_B)
19
20     r_BX = r_AB/(R_A/R_B-1)
21     r_X = r_B*(1+r_BX/a_B)
22     r_PX = np.linalg.norm(r_X - r_P)
23
24     theta_crit = np.arcsin(R_B/r_BX)
25     theta = np.arccos((r_BX**2+r_PX**2-r_BP**2)/(2*r_BX*r_PX))
26
27     T_A = 5913
28     T_B = 5915
29     L_AP = sigma*np.pi*r_P**2*(1-0.3)*(R_A**2*T_A**4/r_AP**2)
30     L_BP = sigma*np.pi*r_P**2*(1-0.3)*(R_B**2*T_B**4/r_BP**2)
31     if theta<=theta_crit:
32         if r_PX<r_BX:
33             L = L_BP
34         else:
35             L = L_AP

```

```

36     else:
37         L = L_AP + L_BP
38
39         T_eq = np.power(L/(4*np.pi*R_P**2*sigma),0.25)
40         I = L/(np.pi*R_P**2*(1-0.3)) # J/(yr*AU**2)
41         radiation_intensity[i] = I / (365.25*86400 * (1.49597871e+11)**2) # W/m^2
42         equilibrium_temperature[i] = T_eq # K
43
44     time = np.linspace(0, t_end, n_steps)
45     fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(12,6), layout='constrained')
46     ax1.plot(time, radiation_intensity)
47     ax1.set_xlabel('Time (yr)')
48     ax1.set_ylabel(r'Radiation Intensity ($W/m^2$)')
49     ax1.set_title('Radiation Intensity over 10 Years')
50     ax2.plot(time, equilibrium_temperature)
51     ax2.set_xlabel('Time (yr)')
52     ax2.set_ylabel('Equilibrium Temperature (K)')
53     ax2.set_title('Equilibrium Temperature over 10 Years')
54     ax1.xaxis.set_minor_locator(AutoMinorLocator())
55     ax1.yaxis.set_minor_locator(AutoMinorLocator())
56     ax2.xaxis.set_minor_locator(AutoMinorLocator())
57     ax2.yaxis.set_minor_locator(AutoMinorLocator())
58     plt.tick_params(which='major',length=5,width=1.5,labelsize='x-large')
59     plt.tick_params(which='minor',length=3)
60     plt.show()
61     fig.savefig('variation.png', format='png', dpi=300, bbox_inches='tight', pad_inches=0.2)

```



如下图所示，辐射强度和平衡温度主要受双星轨道运动周期以及行星绕系统质心轨道运动的周期共同调制。由于我们对遮挡问题的考虑过于简化，因此辐射强度和平衡温度变化图像中出现很陡地突然下降，即由双恒星共同贡献辐射转为某个恒星单独提供辐射，实际上这个过程应该更为平滑，从外切到内切应有较连续的变化。不过仅从目前的简单模型，我们推断出的平衡温度变化已经较好的符合我们在第一问得到的 $251K \sim 348K$ 的结果。此外，辐射强度约为太阳常数 $1367W/m^2$ 的两倍，这也比较合理，因为该行星的半长轴也在 $1AU$ 附近，但是有两颗温度与太阳接近的恒星共同影响。

